Apuntes del Sprint 6 · Datos listos para modelar: pandas, limpieza, pipelines y métricas

LA IDEA CENTRAL DEL SPRINT

un modelo es tan bueno como los datos con los que se alimenta, y solo sabrás si es bueno si lo mides **con honestidad**. Entre el 60 y el 80 % del trabajo real de un proyecto de datos es **entender y limpiar los datos**, no el modelo.

Las diapositivas del Sprint 6 enseñan muchísimo código en muy poco espacio. Estos apuntes **explican qué problema resuelve cada herramienta y por qué se hace en ese orden**, con ejemplos pequeños que puedes comprobar a mano. Léelos antes de cada cuaderno.

0. De qué va el sprint (en 1 minuto)

```
EXPLORAR  →  LIMPIAR  →  PREPARAR  →  MEDIR
(pandas)    (fichero    (pipeline)   (baseline, métricas,
             sucio)                   validación cruzada)
```

Presentación	Pregunta	Cuaderno
S06.1 pandas y EDA	¿Qué hay en estos datos?	S06_01
S06.2 Limpieza y preprocesado	¿Cómo los dejo listos sin hacer trampas?	S06_02, S06_03
S06.3 Métricas y validación	¿Mi modelo es bueno *de verdad*?	S06_04

El reto: el cliente os pasa un fichero **sucio**; hacéis el **inventario de defectos**, una **función de limpieza reproducible**, un **pipeline**, un **baseline** y un informe honesto.

DÍA 1 · Explorar los datos con pandas (EDA)

1. Qué es pandas y qué es un DataFrame

pandas es la librería de Python para trabajar con **tablas**. Un **DataFrame** (`df`) es una tabla: **cada fila es una reserva, cada columna una característica**. En el sprint trabajas con `reservas_turisdata.csv` : **6.000 reservas**, y la columna `cancelada` (0/1) como **objetivo**.

> **Regla de oro: primero mide, luego decide.** Antes de limpiar, modelar o graficar, **mira**.

2. Los cuatro primeros comandos, siempre

```
import pandas as pd
df = pd.read_csv("reservas_turisdata.csv")
df.shape        # (6000, n_columnas): ¿cuántas filas y columnas?
df.info()       # tipo de cada columna y cuántos valores NO nulos tiene
df.head()       # las primeras 5 filas
df.describe()   # resumen de las columnas numéricas (media, mínimo, máximo…)
```

Con ellos respondes: *¿cuántos datos hay? ¿de qué tipo es cada columna? ¿faltan valores? ¿hay valores raros?* Si `info()` dice que `precio_noche` es `object` (texto) en vez de `float64` , **algo está mal** (quizá lleva comas decimales).

3. Filtrar y agrupar

```
df[df["canal"] == "directo"]                          # filtro: solo reservas directas
df[(df["noches"] > 7) & (df["tarifa"] == "flexible")]     # dos condiciones (y = &)
tasa_por_deposito = df.groupby("deposito")["cancelada"].mean()
```

- Cada condición va **entre paréntesis**; `&` = y, `|` = o.
- **La media de una columna 0/1 es una tasa.** Con 4 reservas `web, directo, web, agencia` y cancelaciones `1, 0, 1, 0`: la tasa por canal es web = 100 %, directo = 0 %, agencia = 0 %, y la global es 2/4 = 50 %.
- `groupby` + media responde a **«¿cancela más un grupo que otro?»**.

4. Unir tablas con `merge`

A veces la información está en **varias tablas** (es el JOIN de SQL):

```
df = df.merge(temporadas, on="mes_llegada", how="left")
```

- `on=` es la **columna común**.
- `how="left"` **conserva todas las reservas** aunque falte la fila en la otra tabla (donde no hay pareja quedará vacío).
- **Comprueba siempre las filas antes y después.** Si pasan de 6.000 a 6.350, la unión **duplicó** filas (la otra tabla tenía claves repetidas). Si bajan, **perdiste** datos.

5. Elegir el gráfico según el dato

Quiero ver...	Gráfico	Ejemplo con TurisData
Una variable numérica	**Histograma**	`precio_noche`
Numérica **por categorías**	**Diagrama de cajas**	`precio_noche` por `tipo_habitacion`
Dos numéricas	**Dispersión**	`antelacion_dias` frente a `precio_noche`
Una categórica	**Barras**	reservas por `canal`

Cómo se lee un diagrama de cajas: la línea del centro es la **mediana**; la caja contiene el 50 % central de los datos; los «bigotes» llegan a los valores normales; **los puntos sueltos son posibles *outliers*** (valores muy alejados).

6. Relaciones y primer chequeo de calidad

```
corr = df.select_dtypes("number").corr()                          # matriz de correlaciones
var_mas_ligada = corr["cancelada"].drop("cancelada").abs().idxmax()
filas_duplicadas = df.duplicated().sum()                          # ¿filas repetidas?
pct_nulos_pais = df["pais_origen"].isna().mean() * 100            # % de vacíos en una columna
```

- La **correlación** va de −1 a +1 y mide **relación lineal**. No prueba causalidad (Sprint 5).
- `isna().mean()` es otro truco: `isna()` da True/False por celda y la **media de True/False es la proporción**.
- **Primer chequeo de calidad:** duplicados, nulos y **rangos imposibles**.

Cómo escribir un «hallazgo para el cliente»

Cada hallazgo = una cifra + una frase que un gerente entienda sin saber Python.

Mal	Bien
«La correlación de `deposito` con `cancelada` es −0,27»	«Los clientes **sin depósito cancelan 2,4 veces más** que los que pagan un depósito total (52 % frente a 22 %)» *(cifras de ejemplo)*

DÍA 2 · Limpiar y preparar sin hacer trampas

7. Nunca limpies a ciegas

El cliente envía un segundo fichero, `reservas_sucio.csv`, **exportado a mano desde varios sistemas**. Reglas de trabajo:

```
crudo = pd.read_csv("reservas_sucio.csv")      # el ORIGINAL NO SE TOCA
crudo.info(); crudo.isna().sum(); crudo.describe()
```

Cada defecto se trata con **cuatro pasos**:

1. **Mide** (diagnóstico: ¿cuántas filas afecta?).
2. **Decide** el tratamiento y **anota por qué**.
3. **Escribe el código.**
4. **Comprueba que mejoró.**

Cada paso crea **una copia nueva** (`d1`, `d2`, `d3`…): así puedes ver **cuánto cambia cada decisión**, y volver atrás.

8. Los defectos del fichero sucio y cómo se arreglan

Defecto	Cómo se arregla
Filas duplicadas	`drop_duplicates()` y cuenta cuántas había
Precios con coma decimal («84,36»)	`str.replace(",", ".")` y `.astype(float)`
Fechas en dos formatos	`pd.to_datetime(..., format=..., errors="coerce")` por grupo
Categorías inconsistentes («Directo», «directo », «DIRECTO»)	`strip().lower().replace(" ", "_")`
Valores imposibles y outliers	Reglas de negocio y `NaN`

Ejemplos que puedes comprobar:

- `"84,36"` → `str.replace(",", ".")` → `"84.36"` → `astype(float)` → `84.36`.
- `["Directo", "directo ", " DIRECTO"]` → tras `strip().lower()` → **las tres son `"directo"`**: ahora `groupby` las trata como **un único** canal. Sin limpiar, saldrían **tres** canales distintos y las cifras estarían mal.

Fechas: no adivines el formato

`2024-11-19` es **formato ISO** (año-mes-día) y `19/11/2024` es **día/mes/año**. Para una cifra como `03/04/2024` no se sabe si es 3 de abril o 4 de marzo: **lee cada grupo con su formato**.

```
iso = pd.to_datetime(d2["fecha_reserva"], format="%Y-%m-%d", errors="coerce")
esp = pd.to_datetime(d2["fecha_reserva"], format="%d/%m/%Y", errors="coerce")
d3["fecha_reserva"] = iso.fillna(esp)
```

`errors="coerce"` convierte en vacío (NaT) lo que no encaja con el formato; `iso.fillna(esp)` rellena esos vacíos con la lectura en el otro formato. Resultado: **las dos lecturas dan la misma fecha** (19 de noviembre de 2024).

Imposibles frente a outliers

	Qué es	Qué hacer
Imposible	Precio negativo, 0 noches, llegada **antes** de la reserva	**Error seguro** → se convierte en `NaN`
Outlier	Precio muy alto pero **posible** (una suite en Navidad)	**Se investiga**, no se borra sin más

```
d5.loc[~d5["precio_noche"].between(1, 1000), "precio_noche"] = np.nan
```

Dice: «las filas cuyo precio NO está entre 1 y 1.000, ponles `NaN`». Y de **consistencia entre columnas**: `fecha_llegada` no puede ser anterior a `fecha_reserva`.

9. Nulos: ¿qué hacemos con ellos?

Antes de rellenar, pregunta por qué falta el dato.

Situación	Qué hacer
Falta al azar	**Imputar**: mediana (numéricas) o moda (categóricas)
Falta por algo (un `pais_origen` vacío puede ser información)	Crear la categoría **«desconocido»**
Faltan demasiados	Si la columna está vacía en su mayoría, **descartarla**

¿Por qué mediana y no media? Con precios `[60, vacío, 80, 200]`, la mediana de los tres conocidos es **80** y la media sería 113,3 (arrastrada por el 200). La mediana **resiste a los extremos**.

El mini-reto del cuaderno: empaquetar todo en **una función** `limpiar_reservas(df)`, reproducible, **sin arreglar el CSV a mano**. Si mañana llega un fichero nuevo, la función lo limpia igual.

10. Separar entrenamiento y prueba

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=SEMILLA)
```

- **El 20 % de los datos no se toca hasta el final:** el modelo nunca lo ve al aprender. Es **el examen** (idea que ya viste en el Sprint 1).
- `stratify=y` mantiene **la misma proporción de canceladas** en los dos trozos.
- `random_state` (la semilla) hace la partición **repetible**.

11. Escalado y codificación

Escalado: variables con rangos muy distintos (`precio_noche` de 50 a 250; `ninos` de 0 a 3) confunden a los modelos basados en distancias. Hay que ponerlas **en la misma escala**.

Escalador	Qué hace	Ejemplo con precios 50, 100, 250
MinMax	Lleva todo a [0, 1]	(x − 50) / 200 → **0 ; 0,25 ; 1**
Standard	Media 0, desviación 1 (z-score)	Lo visto en el Sprint 5

Codificación: los modelos solo entienden números, así que las categorías hay que convertirlas.

Tipo	Cuándo	Ejemplo
Ordinal	Las categorías **tienen orden**	`deposito`: sin_deposito=0, parcial=1, total=2
One-hot	**No hay orden**	`canal`: una columna 0/1 por cada canal

One-hot con 4 reservas `web, directo, web, agencia`:

	agencia	directo	web
reserva 1	0	0	1
reserva 2	0	1	0
reserva 3	0	0	1
reserva 4	1	0	0

> **¿Por qué no numerar los canales 1, 2, 3?** Porque el modelo pensaría que `web` (3) es «más que» `agencia` (1). One-hot evita inventar un orden que no existe. `handle_unknown="ignore"` evita que falle si en producción aparece un canal que nunca vio.
>
> **Regla de oro: `fit` solo con entrenamiento; `transform` con entrenamiento y prueba.** El escalador «aprende» la media y el rango **solo de los datos de entrenamiento**.

12. `ColumnTransformer` y `Pipeline`: la receta completa

Cada tipo de columna necesita su tratamiento; todo queda en **un solo objeto** que se ajusta y se aplica **siempre igual**:

```
preprocesado = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                      ("esc", StandardScaler())]), columnas_num),
    ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                      ("oh",  OneHotEncoder(handle_unknown="ignore"))]), columnas_cat),
])
modelo = Pipeline([("prep", preprocesado),
                   ("clf", LogisticRegression(max_iter=1000))])
```

Cómo leerlo, de dentro hacia fuera:

- Columnas **numéricas** → imputar con la **mediana** → escalar.
- Columnas **categóricas** → imputar con la **más frecuente** → one-hot.
- Después, el **clasificador** (regresión logística).

Analogía: una **cadena de montaje**. La materia prima (tabla sucia) entra por un lado y sale el resultado por el otro, siempre por los mismos pasos y en el mismo orden.

13. Fuga de datos en el preprocesado

Hay fuga cuando el modelo «ve» información del conjunto de prueba durante el aprendizaje.

Error típico: escalar **todo** el dataset y **después** partir en train/test. La media y la desviación **ya incluyen** las filas de prueba: el modelo se ha «chivado» sin darse cuenta.

Solución: meter el escalado **dentro del** `Pipeline`. Así se aprende **solo con el trozo de entrenamiento** (y también dentro de la validación cruzada).

> **Mini-reto: ¿es reproducible tu pipeline?** Con la **misma semilla**, debe dar **las mismas predicciones**.

DÍA 3 · Medir con honestidad: scikit-learn

14. El patrón `fit` / `predict` y el baseline

Todos los modelos de scikit-learn siguen **el mismo patrón**:

```
modelo.fit(X_train, y_train)                    # APRENDE
pred  = modelo.predict(X_test)                  # predice la clase (0 / 1)
proba = modelo.predict_proba(X_test)[:, 1]      # probabilidad de ser clase 1
```

Baseline: un modelo «tonto» (`DummyClassifier`: siempre predice la clase más frecuente). **Si tu modelo no lo bate, no sirve.**

15. Métricas de clasificación

Con **positivo = cancelada**. Con 20 reservas (5 canceladas) y un modelo que deja esta matriz:

	Predice: no cancela	Predice: cancela
Real: no cancela (15)	VN = 9	FP = 6
Real: cancela (5)	FN = 1	VP = 4

Métrica	Pregunta que responde	Cálculo
Accuracy	¿Qué proporción acierta?	(4 + 9) / 20 = **65 %**
Precisión	De las que digo «cancelará», ¿cuántas lo hacen?	4 / (4 + 6) = **40 %**
Recall	De las que cancelan, ¿cuántas detecto?	4 / (4 + 1) = **80 %**
F1	Equilibrio entre precisión y recall	2·0,4·0,8 / (0,4 + 0,8) = **0,53**

El baseline «nunca cancela» acierta 15/20 = **75 %** de accuracy… **y detecta 0 cancelaciones** (recall 0). **El modelo tiene peor accuracy y es mucho más útil.** Por eso la accuracy sola **engaña** cuando hay desbalance.

> **¿Cuál priorizar?** Si **perder** una cancelación es caro (habitación vacía) → mira el **recall**. Si cada alerta cuesta dinero (descuentos) → mira la **precisión**. El **F1** equilibra ambas.

AUC-ROC: ¿ordena bien por riesgo?

Mide **si el modelo pone más arriba a las reservas que cancelan que a las que no**, sin fijar umbral. Intuición con 4 reservas: reales `[0, 0, 1, 1]` y probabilidades `[0,1 ; 0,4 ; 0,35 ; 0,8]`. Hay 4 parejas (una cancelada, una no):

Pareja (cancelada vs no)	¿La cancelada tiene mayor probabilidad?
0,35 frente a 0,1	
0,35 frente a 0,4	
0,8 frente a 0,1	
0,8 frente a 0,4	

AUC = 3/4 = **0,75**. **1 = ordena perfecto; 0,5 = como tirar una moneda.**

16. Métricas de regresión

Cuando predices un **número** (por ejemplo, `precio_noche`):

Métrica	Qué dice
MAE	Error medio **en euros**: «nos equivocamos 12 €»
RMSE	Como el MAE pero **castiga más los errores grandes**
R^2	**1 = perfecto; 0 = no mejora a predecir la media**

Ejemplo: precios reales `[100, 120, 90, 110]` y predicciones `[110, 115, 100, 100]`. Errores: 10, 5, 10, 10.

```
MAE  = (10 + 5 + 10 + 10) / 4 = 8,75 €
RMSE = √((100 + 25 + 100 + 100) / 4) = √81,25 ≈ 9,01 €
```

Compara siempre con predecir la media (105 €): sus errores son 5, 15, 15, 5 → MAE = **10 €**. Tu modelo (8,75 €) mejora un poco; su **R^2 = 0,35**.

17. Sobreajuste y sesgo-varianza

Ya lo conoces del Sprint 1:

- **Sobreajuste:** modelo muy flexible que **memoriza** el entrenamiento.
- **Infraajuste:** modelo demasiado simple que **no aprende lo importante**.

En el cuaderno haces crecer la **profundidad de un árbol** y miras el acierto:

Profundidad	Acierto en entrenamiento	Acierto en prueba
1	70 %	69 %
4	82 %	80 %
10	98 %	72 %

(Valores ilustrativos.) Entrenamiento **sigue subiendo**; prueba **sube y luego baja**. **El punto bueno está en medio** (profundidad 4): es la que elige `mejor_profundidad`.

18. Validación cruzada

Problema: una sola partición train/test depende de la **suerte** (¿tocaron las reservas fáciles o las difíciles?).

Solución: dividir en **5 trozos**; entrenar con 4 y evaluar con el 5.º, **5 veces**, rotando. Obtienes **5 notas** y su variabilidad.

```
auc_cv = cross_val_score(logistica, X_train, y_train,
                         cv=StratifiedKFold(5, shuffle=True, random_state=SEMILLA),
                         scoring="roc_auc")
auc_cv.mean(), auc_cv.std()
```

Resultado	Cómo leerlo
media 0,78, desv. 0,01	**Estable**: el resultado no depende de la suerte
media 0,78, desv. 0,10	**Inestable**: podría ser 0,68 o 0,88 según la partición

- Se hace **con el `Pipeline` completo**, para que el preprocesado no filtre información.
- **El test final se usa una sola vez, al terminar.**

> **Mini-reto:** tabla `resultados` con `auc_cv_media` y `auc_cv_desv` del **baseline** y de la **logística**, y una frase `conclusion`: **¿compensa el modelo?**

19. Fuga de datos: la prueba del «¿lo sabía al reservar?»

Hay fuga cuando el modelo usa información que no tendría en el momento de predecir.

Columna candidata	¿Se conoce al reservar?
`antelacion_dias`	Sí: días entre reservar y llegar
`tarifa`	Sí: se elige al reservar
`cancelaciones_previas`	Sí: historial del cliente
`importe_devuelto`	**No**: euros reembolsados *tras* cancelar → **fuga**
`fecha_cancelacion`	**No**: vacía si no canceló → **fuga**

> **La primera señal de fuga: un AUC «demasiado bueno»** (0,99). Si de repente tu modelo es casi perfecto, **desconfía y revisa las columnas**, en vez de celebrarlo. En el ejercicio rellenas `columnas_con_fuga`.

---

20. Dónde practicar cada cosa

Si no entiendes...	Mira la sección	Practica en
`info`, `describe`, filtros, `groupby`, `merge`, gráficos	1-6	Cuaderno `S06_01_pandas_eda`
Duplicados, fechas, categorías, imposibles, nulos	7-9	`S06_02_limpieza`
Train/test, escalado, one-hot, pipeline	10-13	`S06_03_preprocesado_pipeline`
Baseline, métricas, validación cruzada, fuga	14-19	`S06_04_metricas_validacion`
Todo junto	todas	Reto: inventario de defectos, función de limpieza, pipeline, baseline, informe

21. Chuleta de vocabulario

Palabra	En lengua normal
EDA	Análisis exploratorio: mirar los datos antes de modelar
DataFrame	Tabla de pandas
NaN / NaT	Valor vacío (número / fecha)
Duplicado	Fila repetida
Outlier	Valor muy alejado del resto, pero posible
Imputar	Rellenar un vacío con un valor razonable
Escalar	Poner las variables en una escala comparable
One-hot	Una columna 0/1 por cada categoría

Palabra	En lengua normal
Pipeline	Cadena de pasos que se aplica siempre igual
Train / test	Datos para aprender / datos para el examen final
Estratificar	Mantener la misma proporción de clases en cada trozo
Baseline	Modelo «tonto» que hay que batir
Precisión / Recall / F1	Cuántos avisos son ciertos / cuántos casos detecto / equilibrio
AUC-ROC	Qué bien ordena el modelo por riesgo (0,5 azar; 1 perfecto)
MAE / RMSE / R^2	Error medio / error que castiga lo grande / mejora frente a la media
Validación cruzada	Evaluar varias veces rotando los trozos
Fuga de datos	El modelo ve información que no tendría al predecir

22. Cómo estudiar estos apuntes

1. **Antes de cada cuaderno**, lee la sección que indica la tabla del apartado 20.
2. **Comprueba a mano los ejemplos** de métricas (matriz → precisión, recall, F1): es lo que más se confunde.
3. **En el reto, lleva un diario de decisiones**: cada defecto, **por qué** lo trataste así y **cuánto cambió**. Eso vale más que el código.
4. **Desconfía de los resultados demasiado buenos**: antes de celebrarlos, busca la fuga.